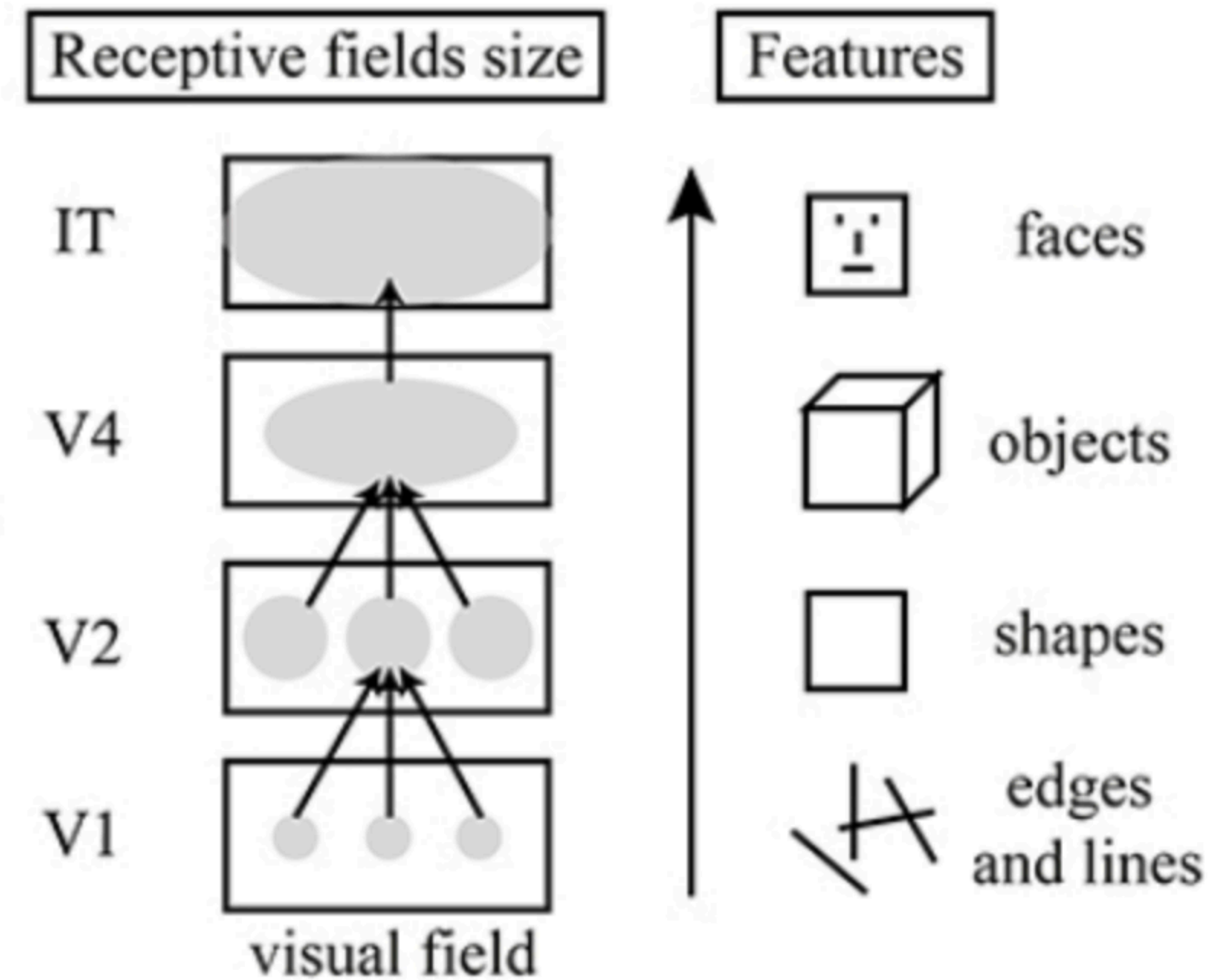
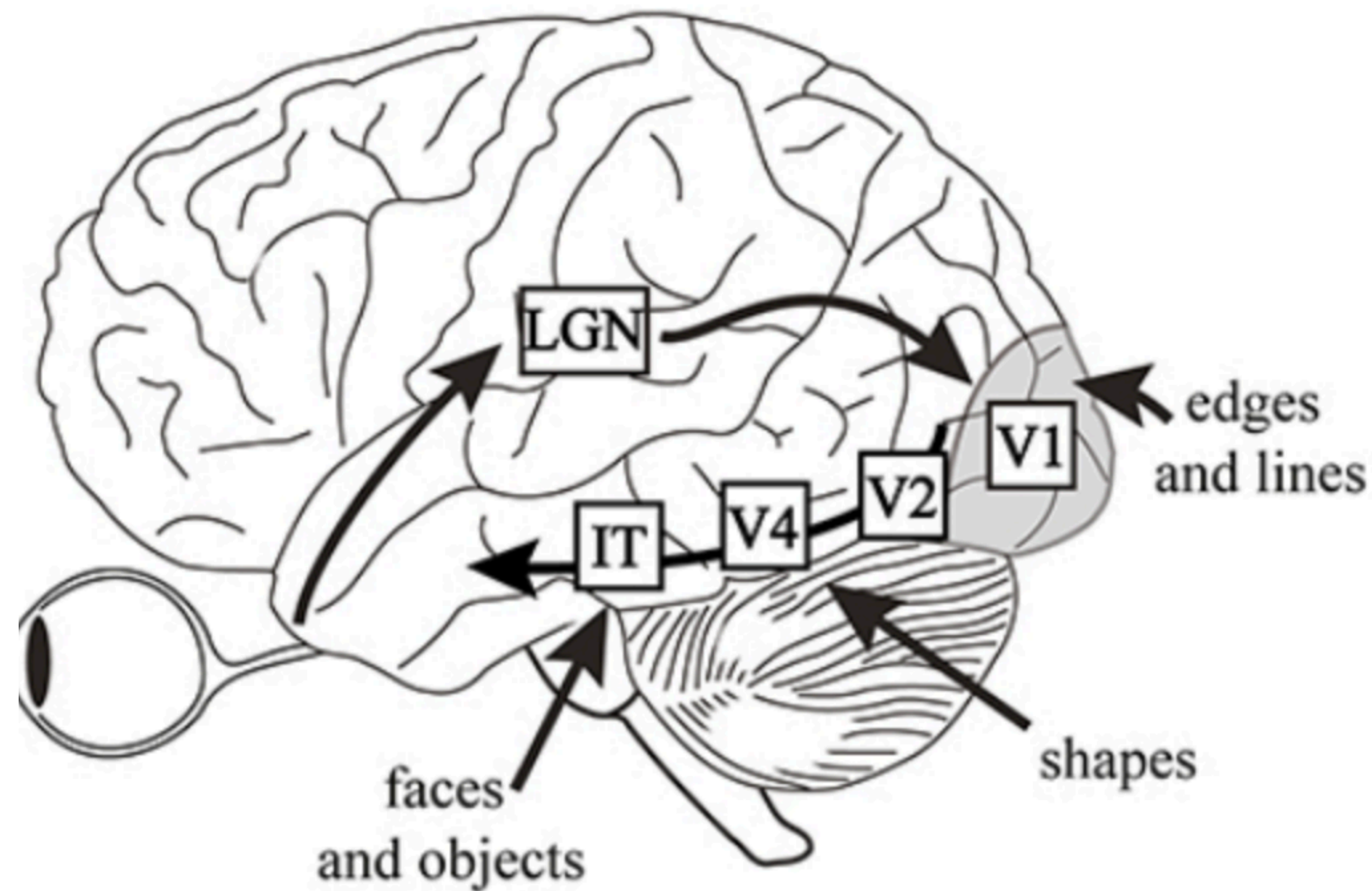


Bazele Inteligenței Artificiale

Lab 9. Rețele neuronale convoluționale

Efrem Dragoș-Sebastian-Mihaly

Procesarea informațiilor vizuale în creierul uman



Procesarea informațiilor vizuale în creierul uman

- Cercetătorii au descoperit că oamenii percep informațiile vizuale într-un mod ierarhic
- Asta înseamnă că nu există un anumit loc în creier care procesează toate informațiile dintr-o dată, ci mai degrabă există anumite părți din creier care procesează datele rând pe rând
- Aceste zone din creier poartă numele de cortexuri
- Mai întâi, lumina este detectată de retina din ochi, iar mai apoi retina transformă lumina în semnale electrice
- Semnalele electrice generate în retină sunt transmise către creier prin nervul optic, sunt filtrate iar mai apoi sunt transmise către cortexul vizual primar

Procesarea informațiilor vizuale în creierul uman

- Cortexul vizual primar (V1) detectează caracteristicile de bază din informațiile vizuale, cum ar fi linii, margini sau schimbări de contrast
- Acest output este transmis către cortexul vizual secundar (V2) care combină informațiile detectate (linii, margini) pentru a identifica forme și contururi mai complexe și are un rol important în percepția culorilor și mișcării
- Mai este și cortexul vizual terciar (V3) și cel V4, care combină caracteristicile mai mult pentru a recunoaște obiecte pe baza forme
- Cortexul vizual inferotemporal (IT) integrează informația vizuală pentru a detecta fețe, animale, obiecte cunoscute
- Scopul nostru e să realizăm modele care simulează percepția vizuală a omului

Simularea percepției vizuale umane

- Pentru a simula percepția umană, avem nevoie de un mod prin care putem identifica ierarhic caracteristici vizuale
- Asta înseamnă să avem o ierarhie: de la un strat neuronal la altul, neuronii integrează caracteristicile de la stratul precedent pentru a identifica lucruri complexe
- Pentru început, avem nevoie de un mod prin care, dintr-o imagine, putem filtra elemente de bază, sau „locale”
- Scopul e ca din mai multe informații locale, să putem extrage informații generale
- O operație ajutătoare în acest sens este operația de convoluție

Ce este convoluția?

- Convoluția reprezintă baza prin care putem extrage informații locale dintr-o imagine
- La început, imaginile au fost alb-negru
- Aceste imagini au fost concepute ca fiind o matrice de valori, iar cu cât valoarea dintr-un punct era mai mare, cu atât acel punct era mai luminos
- Într-o astfel de imagine simplă, alb-negru, convoluția se uită peste valorile acelei matrice, „scanând”, iar mai apoi rezultă o nouă matrice cu informații extrase din cea originală

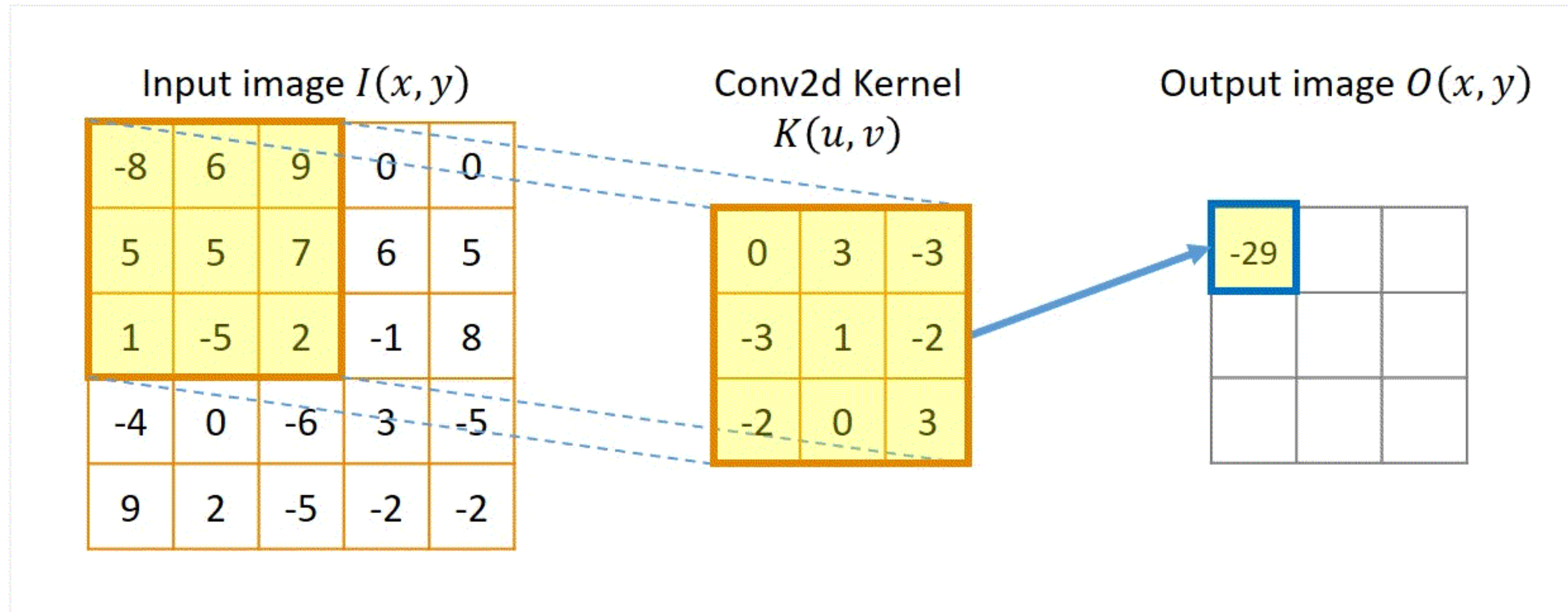


Ce este convoluția?



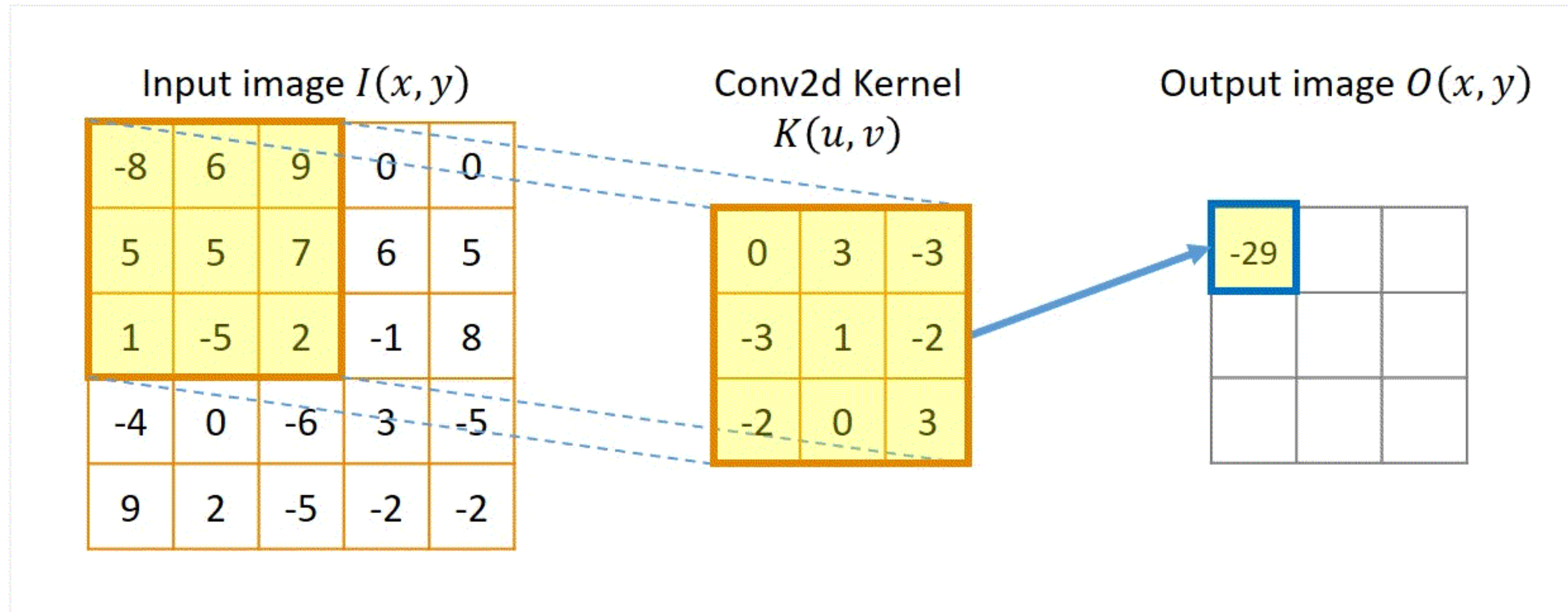
- Putem observa că matricile noi obținute după procesul de convoluție evidențiază anumite aspecte (cum ar fi margini)
- Asta reprezintă partea de identificare a caracteristicilor locale despre care vorbeam mai devreme
- Și chiar dacă unele dintre acestea poate par inutile sau slabe, scopul nostru este să identificăm cât mai multe informații locale ca să avem o intuiție globală

Ce este convoluția?



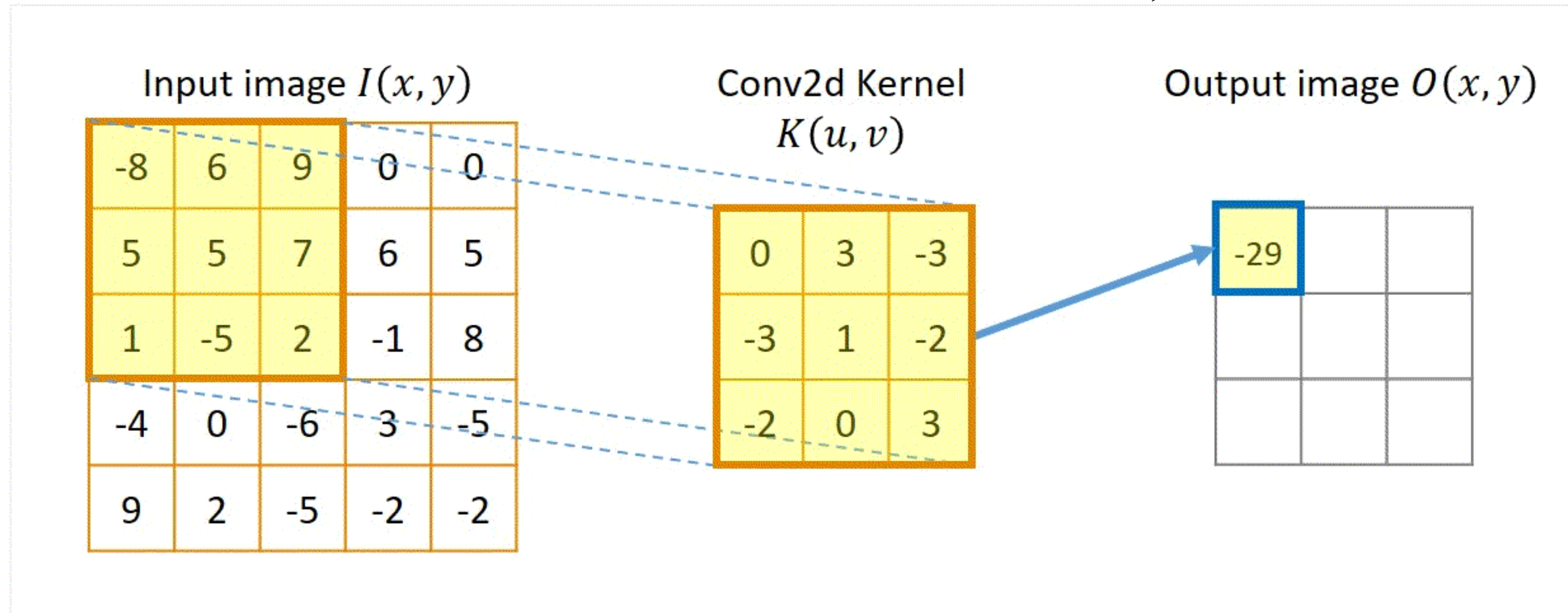
- Convoluția presupune aplicarea unui filtru (kernel) care se ocupă cu această parte de scanare
- Prima dată filtrul este așezat în stânga sus ca în imagine
- Prima valoare rezultată în output este dată de produsul elementelor de pe pozițiile corespunzătoare și adunarea lor

Ce este convoluția?



- Mai sus, valoarea -29 se obține ca: $(-8) * 0 + 6 * 3 + 9 * (-3) + 5 * (-3) + 5 * 1 + 7 * (-2) + 1 * (-2) + (-5) * 0 + 2 * 3 = -29$

Ce este convoluția?



- Se continuă aplicarea filtrului ca mai sus, filtrul deplasându-se la dreapta și după în jos
- De aceea spunem că filtrul scanează, deoarece se uită peste toate porțiunile din matricea care se potrivesc cu dimensiunile filtrului în stilul de mai sus

Ce este convoluția?

```
▶ import torch
import torch.nn.functional as F

X = torch.tensor([[[[0.0, 1.0, 2.0],
                    [3.0, 4.0, 5.0],
                    [6.0, 7.0, 8.0]]]])

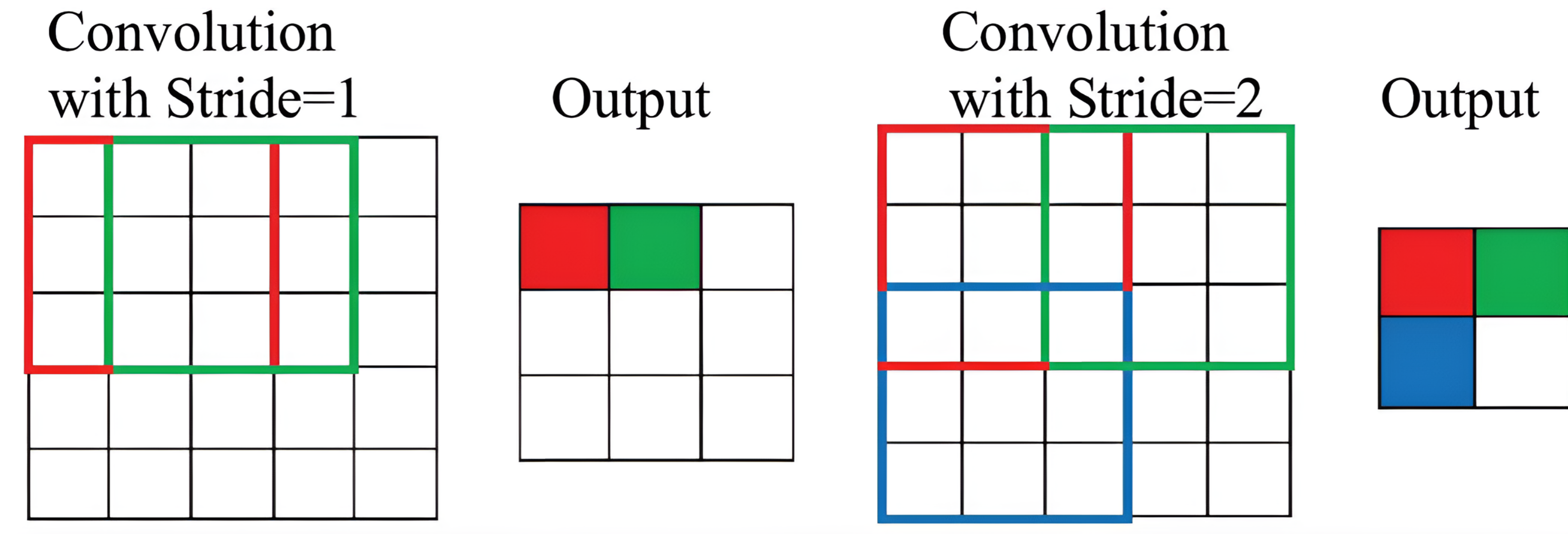
K = torch.tensor([[[[0.0, 1.0],
                    [2.0, 3.0]]]])

output = F.conv2d(X, K)

print(output)
```

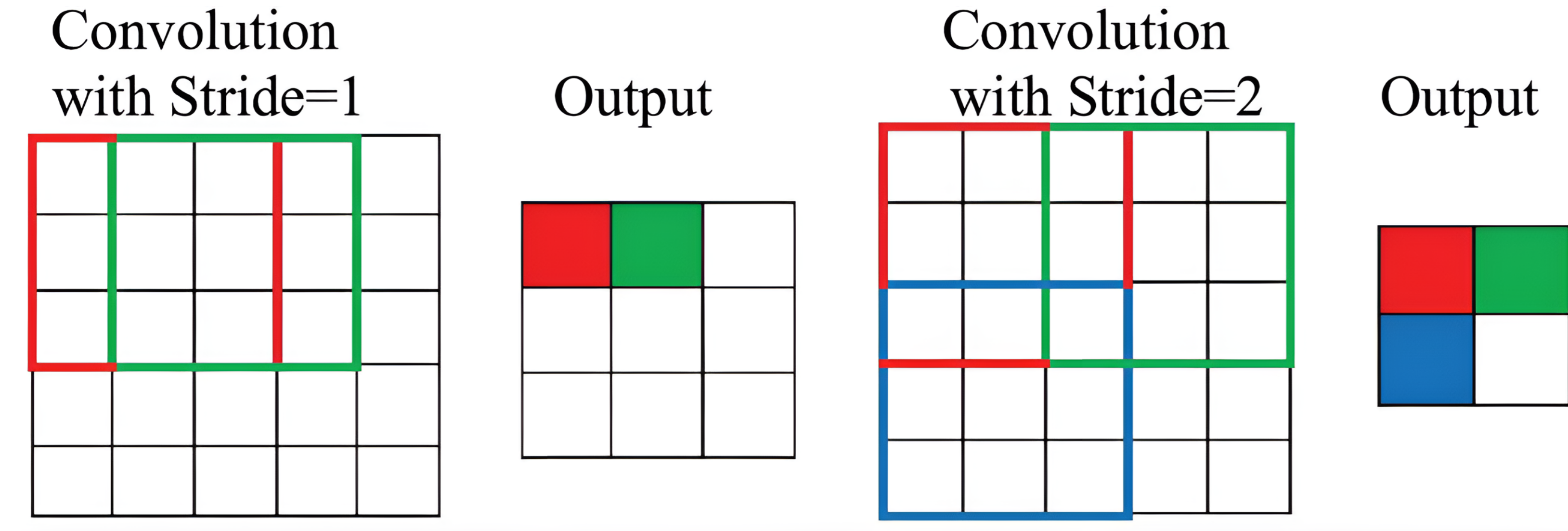
```
⇒ tensor([[[[19., 25.],
            [37., 43.]]]])
```

Stride și Padding



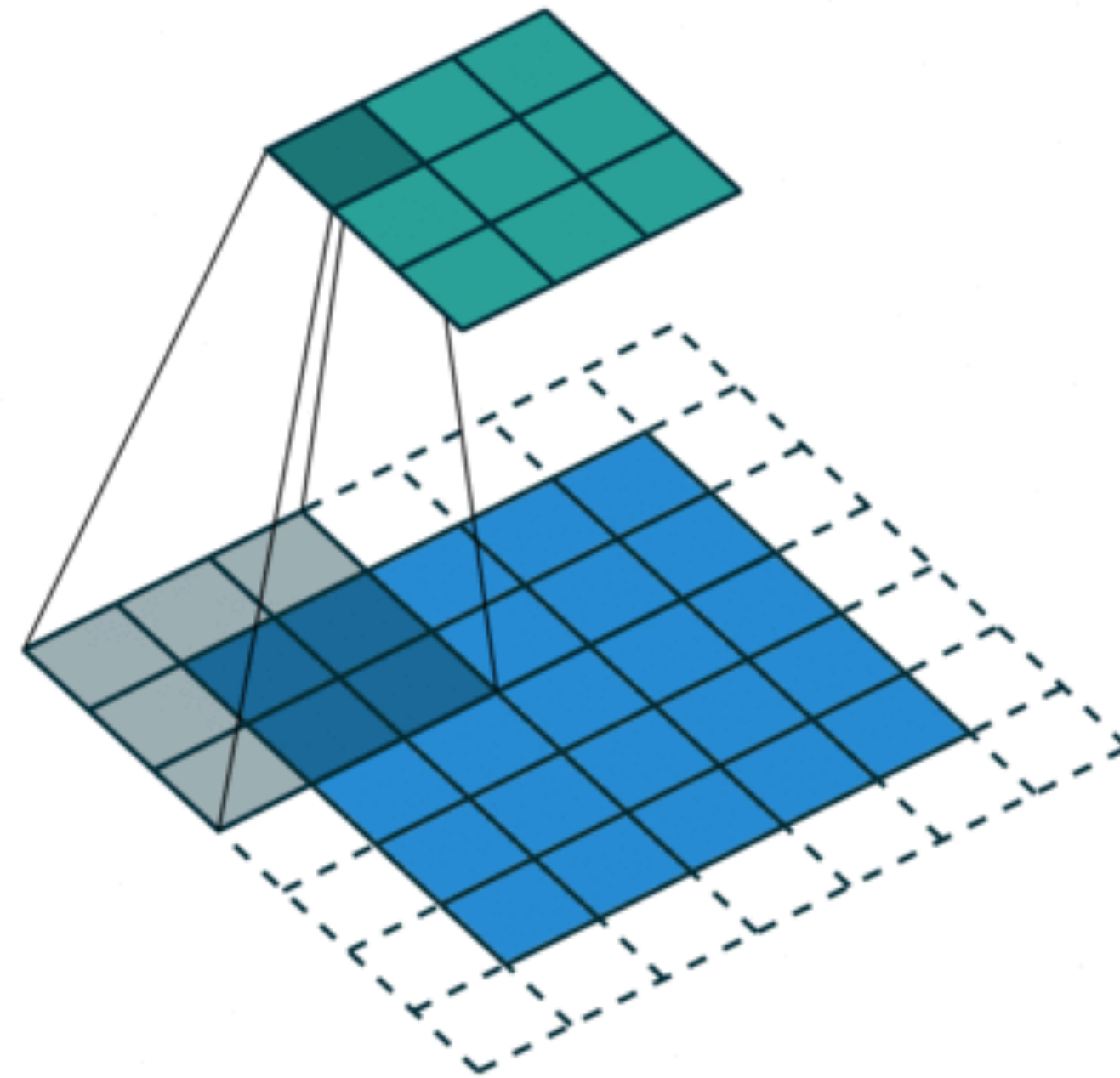
- Stride-ul spune din câți în câți pixeli se obține celulele din matricea de ieșire
- Exemplu:
 - Stride = 1: Kernelul se deplasează câte un pixel pe rând și pe coloană
 - Stride = 2 sau mai mare: Kernelul „sare” peste poziții, deplasându-se câte doi sau mai mulți pixeli pe rând și pe coloană

Stride și Padding



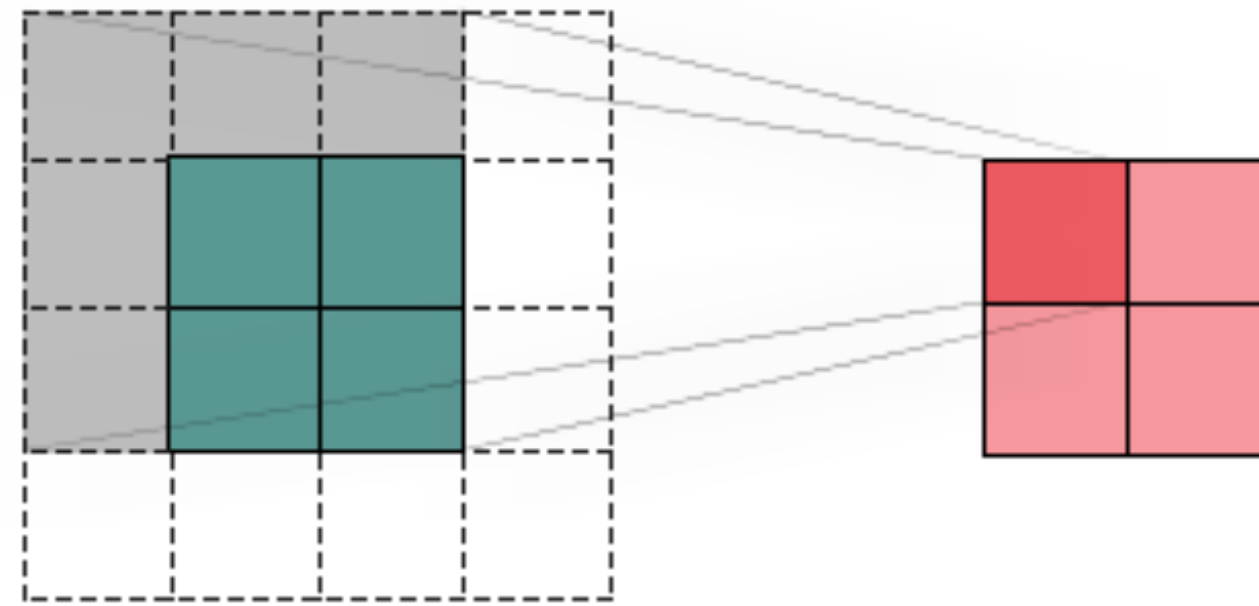
- Un stride mai mic analizează imaginea mai în detaliu
- Un stride mare sare peste detalii
- ieșirea reprezintă tot detalii locale, dar cu o granularitate mai mică (stride mare) sau mai mare (stride mic)

Stride și Padding



- Padding adaugă margini artificiale în jurul datelor de intrare
- În absența padding-ului, părțile din imagine aflate la margini sunt procesate mai puțin sau chiar pierdute
- Padding asigură că toți pixelii contribuie în mod egal la obținerea informațiilor locale

Stride și Padding



- Mai este folosit și pentru a nu reduce dimensiunea matricei de intrare în urma convoluției (păstrăm mai multă informație)
- De exemplu, sus se aplică un filtru de 3x3 asupra unei imagini de 2x2, cu padding, rezultatul este tot o matrice 2x2

Stride și Padding

- Stride:

```
X = torch.rand(1, 1, 8, 8)
# Note that here 1 row or column is padded on either side, so a total of 2
# rows or columns are added
conv2d = nn.Conv2d(1, 1, kernel_size=3, padding=1)
#conv2d = nn.Conv2d(1, 1, kernel_size=(3,3), padding=(1,1))
conv2d(X).shape

torch.Size([1, 1, 8, 8])
```

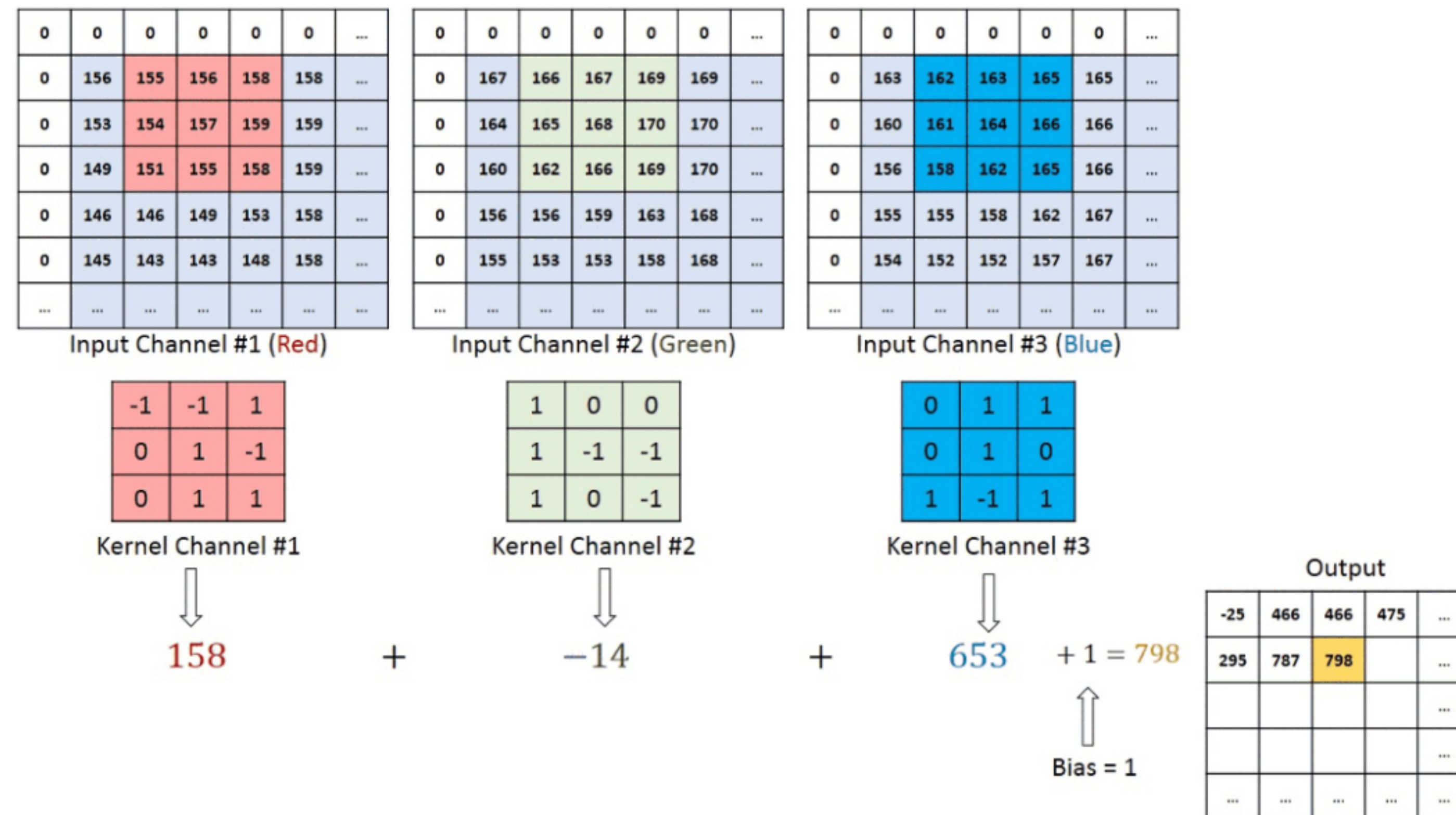
- Padding:

```
X = torch.rand(1, 1, 8, 8)
conv2d = nn.Conv2d(1, 1, kernel_size=3, padding=1, stride=2)
conv2d(X).shape

torch.Size([1, 1, 4, 4])
```

Ce este convoluția?

- Imaginile din ziua de azi au de obicei 3 canale (unul pentru fiecare dintre culorile roșu, verde și albastru)
- Cele 3 canale vor fi reprezentate prin 3 matrici care ne ajută să reprezentăm orice culoare posibilă
- În acest caz, vom folosi un filtru cu 3 canale
- Valoarea finală dintr-o anumită celulă din matricea rezultată se obține ca suma rezultatelor intermediare corespunzătoare fiecărui filtru

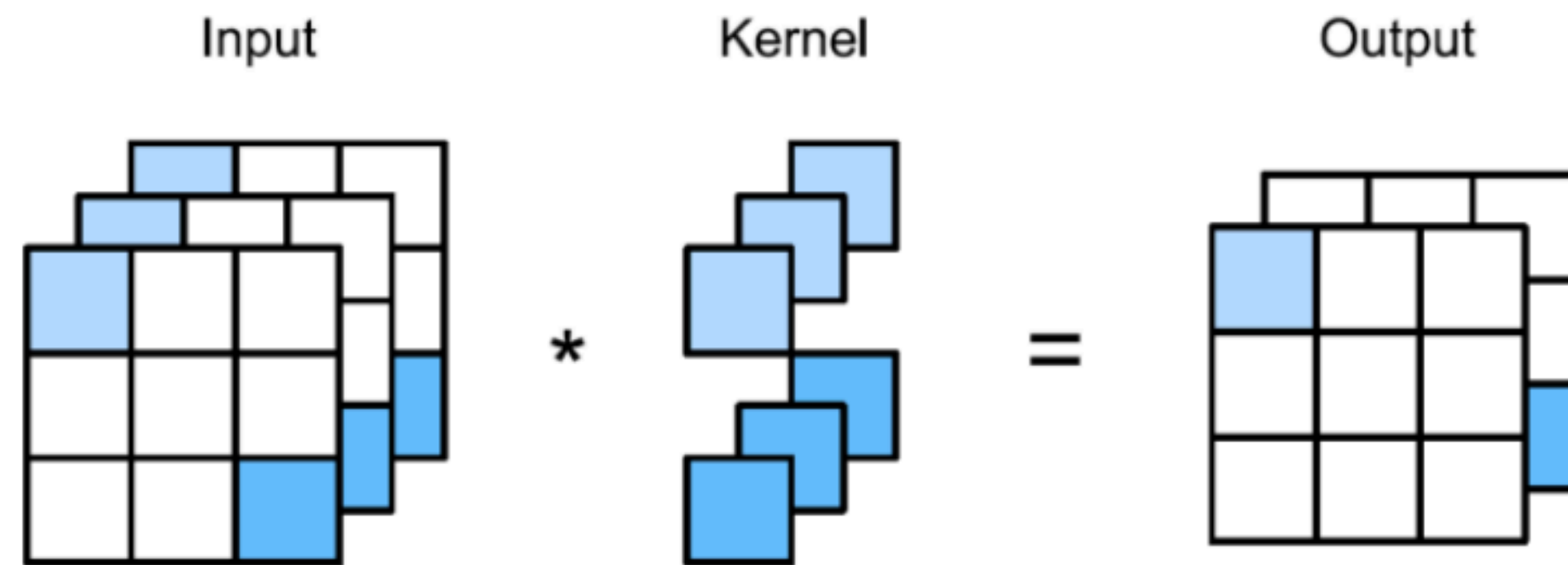


Ce este convoluția?

- Ideea de bază este că dacă avem o intrare cu N canale, vom avea un filtru cu N canale pentru a produce o singură matrice de ieșire
- Rolul acelei matrici de ieșire este să captureze informație locală din toate canalele
- Această matrice de ieșire mai poartă numele de hartă de caracteristici (feature map)
- Oare dintr-un input, este de ajuns să ajungem la un singur feature map?
- Dacă am avea mai multe feature maps, am captura mai multă informație locală
- Aceste informații locale pot fi combinate cumva mai în colo să obținem informații globale

Ce este convoluția?

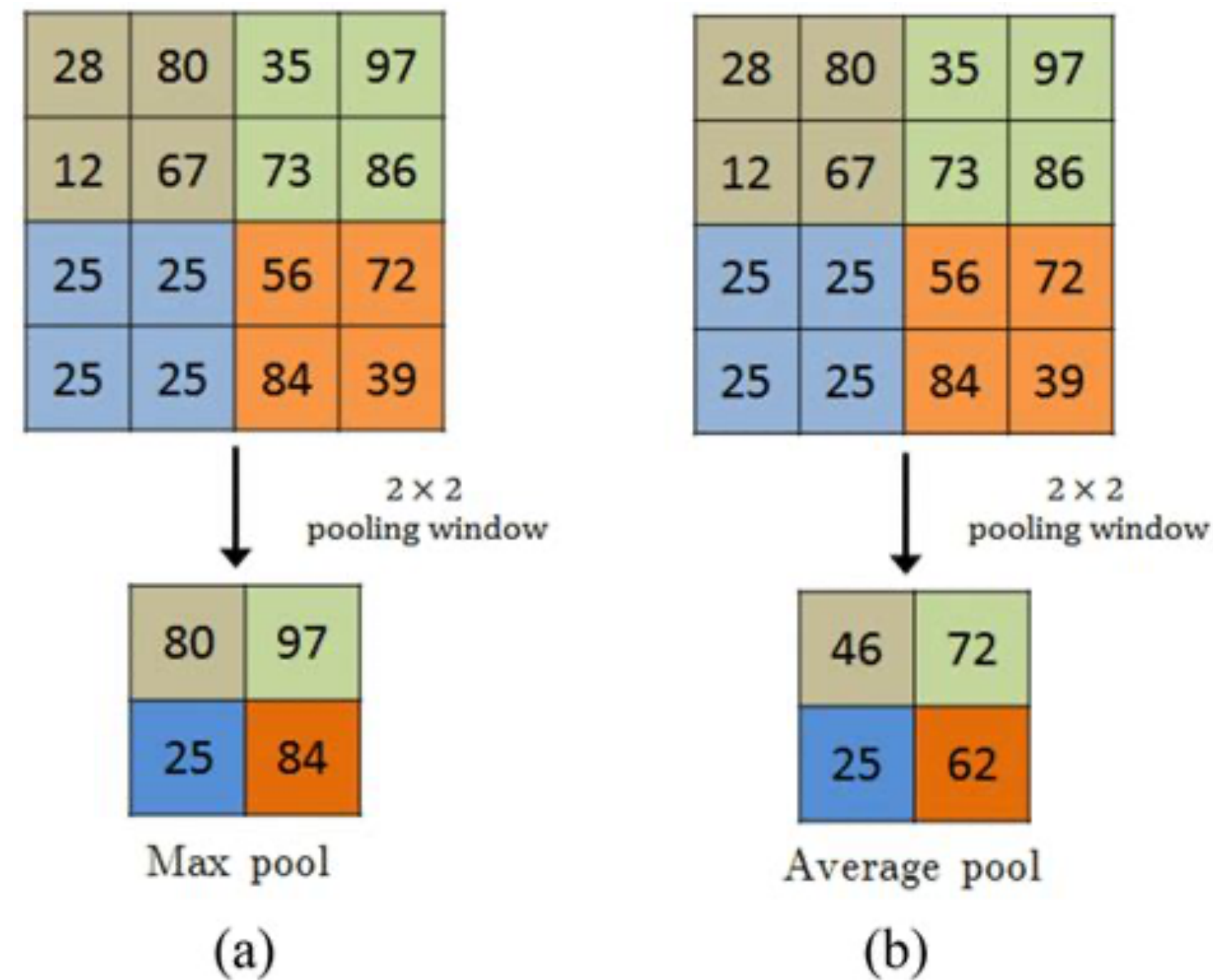
- Rezolvăm problema aplicând mai multe filtre:



- Spre exemplu, pentru o imagine RGB ca cea de mai sus, ca să obținem două feature maps, atunci aplicăm două filtre diferite, fiecare cu 3 canale
- Astfel, obținem două feature maps, fiecare dintre ele capturând diferit informațiile locale din imagine
- În cazul de mai sus, se mai spune și că se trece de la 3 canale la 2 canale, pentru că obținem 2 feature maps care poate fi considerat un singur feature map cu două canale

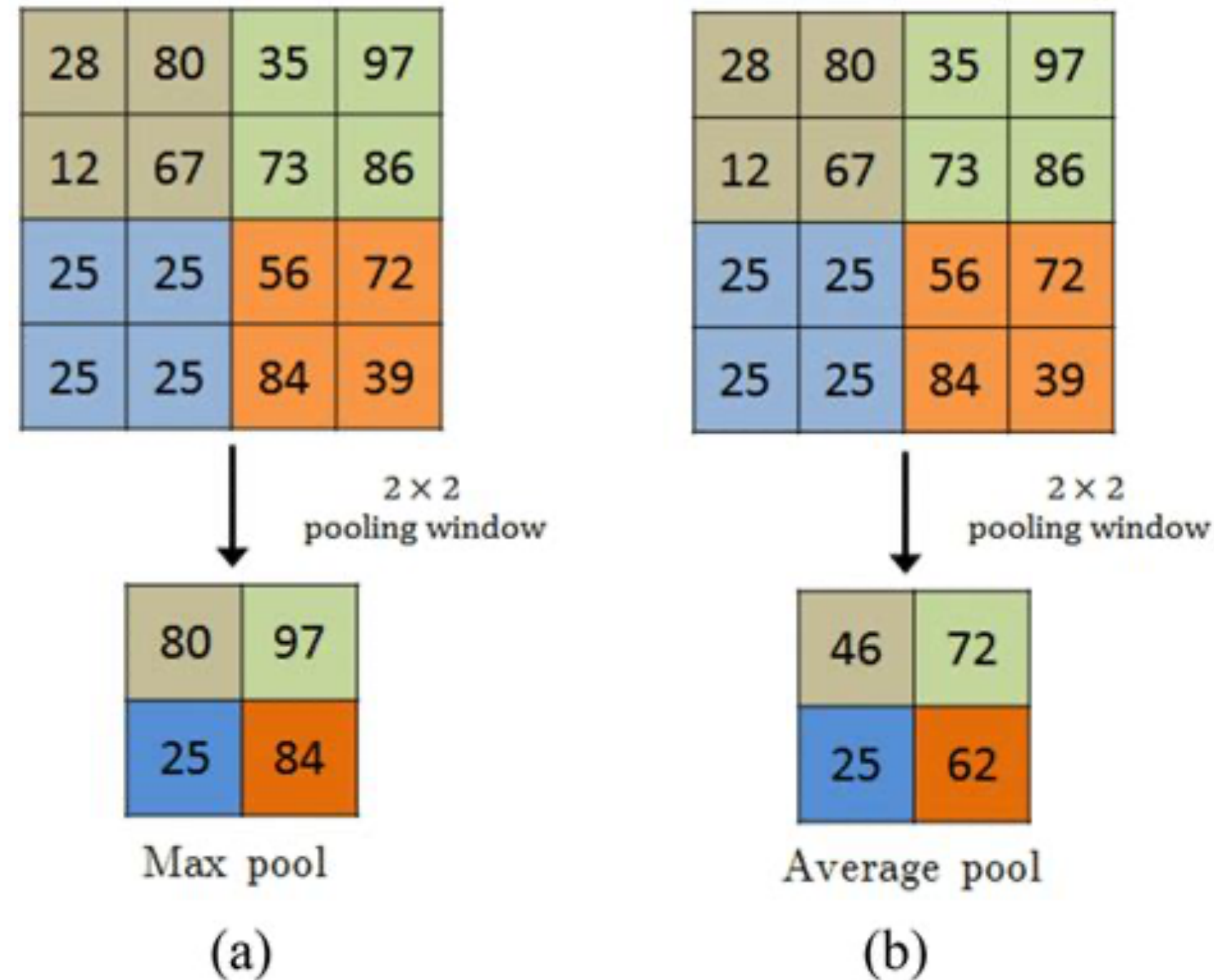
Operația de Pooling

- Pooling-ul este o operație care „rezumă” datele dintr-o matrice de intrare



- Sunt două tipuri des folosite: max pooling și average pooling

Operația de Pooling



- Max pooling: alege cea mai mare valoare dintr-o fereastră
- Average pooling: calculează media valorilor dintr-o fereastră
- Prin pooling, se reduce sensibilitatea la mici modificări sau translații ale imaginii (se reduce impactul zgomotului)

Operația de Pooling

- Aplicarea pooling pe o imagine:

(446, 450)



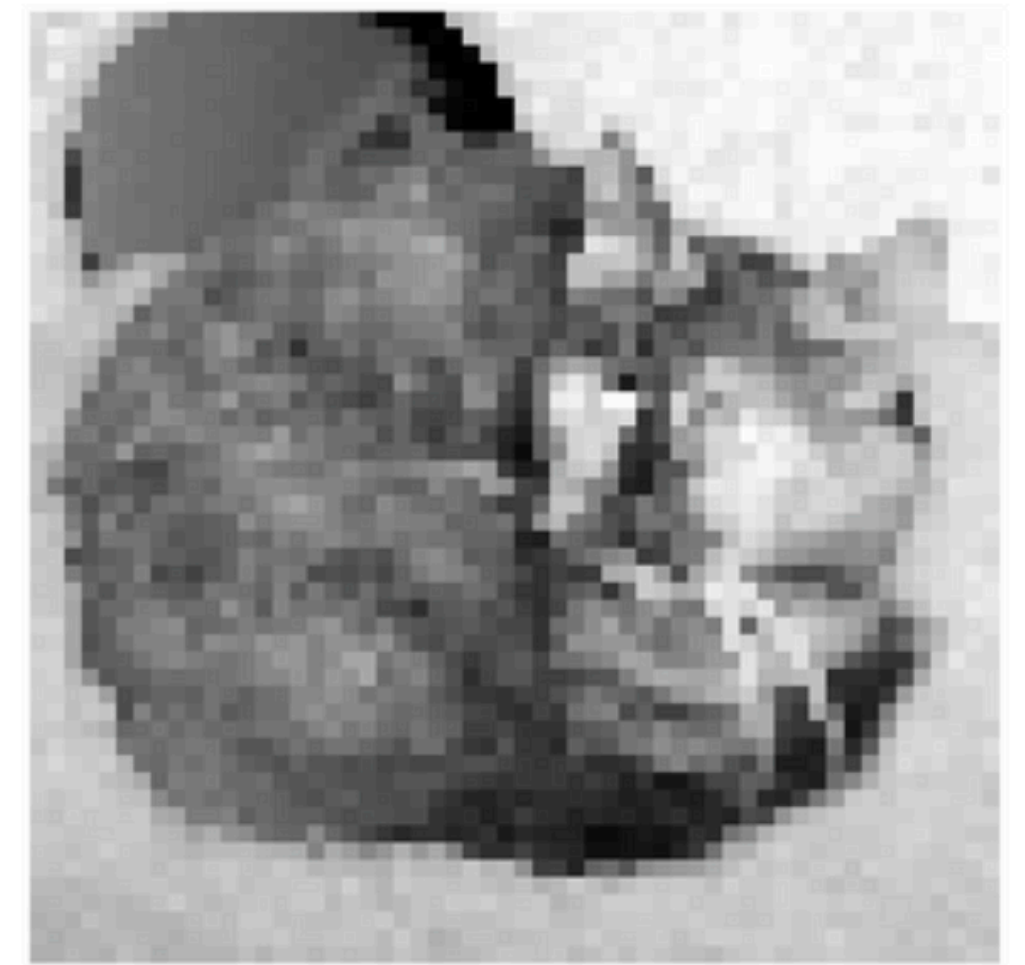
(223, 225)



(111, 112)



(55, 56)



Operația de Pooling

```
X = torch.arange(16, dtype=torch.float32).reshape((1, 1, 4, 4))  
print(X)
```

```
pool2d = nn.MaxPool2d(3)  
pool2d(X)
```

```
tensor([[[[ 0.,  1.,  2.,  3.],  
           [ 4.,  5.,  6.,  7.],  
           [ 8.,  9., 10., 11.],  
           [12., 13., 14., 15.]]]])  
tensor([[[[10.]]]])
```

- În pyTorch, MaxPool2d(k) primește dimensiunea „ferestrei” de scanare: mai sus e 3, deci fereastra va avea dimensiunea 3x3
 - Tot în PyTorch, default, stride = k dacă nu este specificat contrar, deci în cazul de mai sus stride = 3
-
- Mai sus, ieșirea este 10, deoarece fereastra se aplică doar o dată (nu se poate deplasa mai în dreapta sau în jos, din cauza stride-ului mare)

Un aspect important..

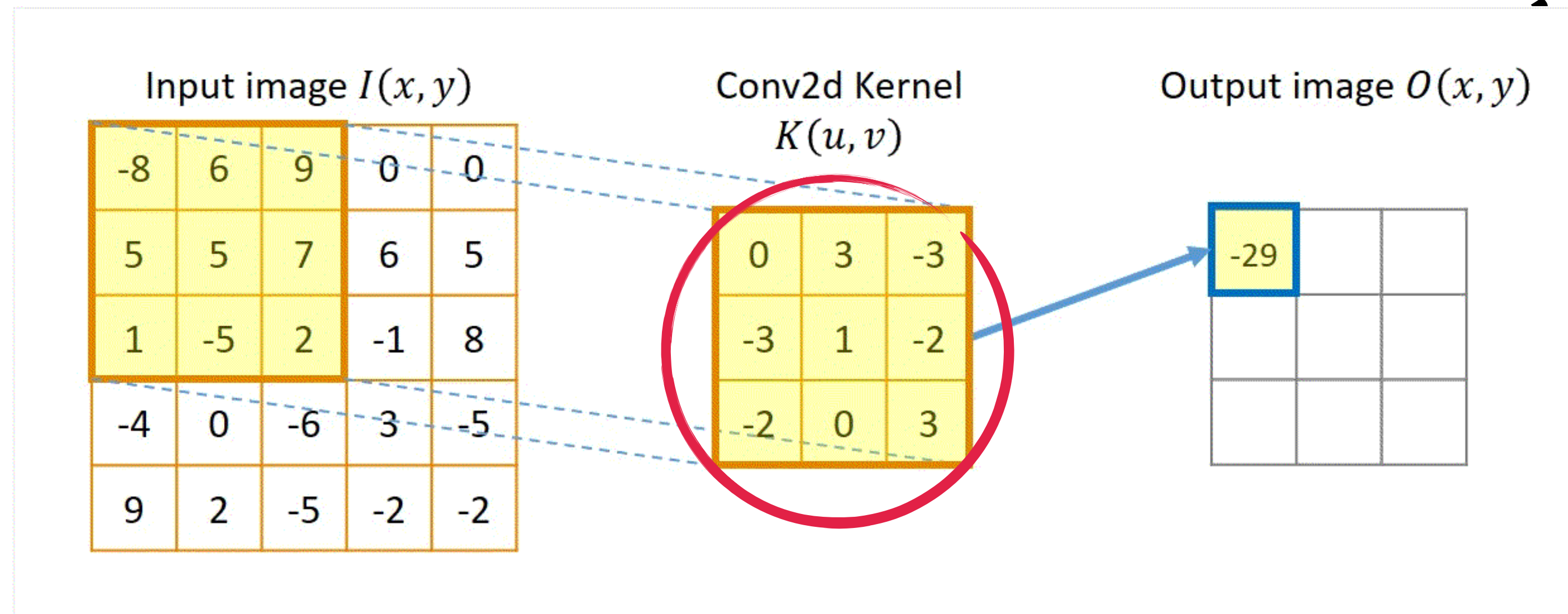
- Tot ce am vorbit până acum este foarte esențial, dar lipsesc răspunsurile la unele întrebări
- Avem operația de convoluție, dar cum știm ce filtru să aplicăm să extragem informații importante?
- După ce aplicăm convoluția obținem informații locale, cum putem combina mai multe informații locale să obținem informații globale?
- Mai exact, cum continuăm acel proces de imitare a structurii ierarhice ale creierului când vine vorba de percepția vizuală?

Răspuns: Rețele neuronale convoluționale

Rețele neuronale convoluționale

- Avem operația de convoluție, dar cum știm ce filtru să aplicăm?

Răspuns: Facem un model care să învețe filtrul



- Asta înseamnă să se învețe care sunt numerele necesare din filtru pentru a învăța informațiile locale din imagine

Rețele neuronale convoluționale

- După ce aplicăm convoluția obținem informații locale, cum putem combina mai multe informații locale să obținem informații globale?

Răspuns: Nu învățăm un singur filtru, învățăm mai multe filtre

Mai multe filtre = mai multe informații locale

Rețele neuronale convoluționale

- Mai exact, cum continuăm acel proces de imitare a structurii ierarhice ale creierului când vine vorba de percepția vizuală?

Răspuns: Folosim mai multe straturi convoluționale

- Fiecare strat va învăța un anumit număr de filtre (N), deci va da ca ieșire N feature maps
- Stratul următor va prelua cele N feature maps, și va aplica și el un anumit număr de filtre, și tot așa
- Practic, se folosesc informațiile locale de la stratul anterior pentru a învăța relații mai globale

Rețele neuronale convoluționale

- De exemplu, o rețea neuronală convoluțională clasică (LeNet) arată așa:

```
net = nn.Sequential(  
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Sigmoid(),  
    nn.AvgPool2d(kernel_size=2, stride=2),  
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),  
    nn.AvgPool2d(kernel_size=2, stride=2),  
    nn.Flatten(),  
    nn.Linear(16 * 5 * 5, 120), nn.Sigmoid(),  
    nn.Linear(120, 84), nn.Sigmoid(),  
    nn.Linear(84, 10))
```

```
X = torch.rand(1, 1, 28, 28)  
for layer in net:  
    X = layer(X)  
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
Conv2d output shape:      torch.Size([1, 6, 28, 28])  
Sigmoid output shape:    torch.Size([1, 6, 28, 28])  
AvgPool2d output shape:  torch.Size([1, 6, 14, 14])  
Conv2d output shape:      torch.Size([1, 16, 10, 10])  
Sigmoid output shape:    torch.Size([1, 16, 10, 10])  
AvgPool2d output shape:  torch.Size([1, 16, 5, 5])  
Flatten output shape:    torch.Size([1, 400])  
Linear output shape:      torch.Size([1, 120])  
Sigmoid output shape:    torch.Size([1, 120])  
Linear output shape:      torch.Size([1, 84])  
Sigmoid output shape:    torch.Size([1, 84])  
Linear output shape:      torch.Size([1, 10])
```

- Observăm că pe măsură ce înaintăm în straturi, scade rezoluția imaginii inițiale (prin convoluție), și crește numărul de canale (de informații locale extrase)
- Se folosește Average Pooling între straturile convoluționale pentru a elimina zgomot
- La final de tot, avem un simplu MLP (mai poartă denumirea de fully connected network), care primește matricile final învățate (16 matrici de 5x5) și ia o decizie

Rețele neuronale convoluționale

- De exemplu, o rețea neuronală convoluțională clasică (LeNet) arată așa:

```
net = nn.Sequential(  
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Sigmoid(),  
    nn.AvgPool2d(kernel_size=2, stride=2),  
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),  
    nn.AvgPool2d(kernel_size=2, stride=2),  
    nn.Flatten(),  
    nn.Linear(16 * 5 * 5, 120), nn.Sigmoid(),  
    nn.Linear(120, 84), nn.Sigmoid(),  
    nn.Linear(84, 10))
```

```
X = torch.rand(1, 1, 28, 28)  
for layer in net:  
    X = layer(X)  
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
Conv2d output shape:      torch.Size([1, 6, 28, 28])  
Sigmoid output shape:    torch.Size([1, 6, 28, 28])  
AvgPool2d output shape:  torch.Size([1, 6, 14, 14])  
Conv2d output shape:      torch.Size([1, 16, 10, 10])  
Sigmoid output shape:    torch.Size([1, 16, 10, 10])  
AvgPool2d output shape:  torch.Size([1, 16, 5, 5])  
Flatten output shape:    torch.Size([1, 400])  
Linear output shape:      torch.Size([1, 120])  
Sigmoid output shape:    torch.Size([1, 120])  
Linear output shape:      torch.Size([1, 84])  
Sigmoid output shape:    torch.Size([1, 84])  
Linear output shape:      torch.Size([1, 10])
```

- De asemenea, observăm folosirea funcției de activare Sigmoid după fiecare strat convoluțional
- În acest caz, funcția de activare permite învățarea de filtre complexe, non-liniare, care pot captura detalii locale mai importante
- Cu cât înaintăm mai adânc în straturi, cu atât modelul combină relații locale ca să învețe relații globale

Structura clasică a unei rețele neuronale convoluționale

```
Sequential(  
  (0): Sequential(  
    (0): Conv2d(1, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): ReLU()  
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (1): Sequential(  
    (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): ReLU()  
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (2): Sequential(  
    (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): ReLU()  
    (2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (3): ReLU()  
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (3): Sequential(  
    (0): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): ReLU()  
    (2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (3): ReLU()  
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (4): Sequential(  
    (0): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): ReLU()  
    (2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (3): ReLU()  
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (5): Flatten(start_dim=1, end_dim=-1)  
  (6): Linear(in_features=25088, out_features=4096, bias=True)  
  (7): ReLU()  
  (8): Dropout(p=0.5, inplace=False)  
  (9): Linear(in_features=4096, out_features=4096, bias=True)  
  (10): ReLU()  
  (11): Dropout(p=0.5, inplace=False)  
  (12): Linear(in_features=4096, out_features=10, bias=True)
```

- O rețea neuronală convoluțională clasică are următoarea structură:

- Un strat convoluțional pentru a învăța N filtre
- O funcție de activare pentru a învăța filtre complexe
- O operație de pooling pentru filtrarea zgomotului
- Acesta este un bloc fundamental
- După folosim mai multe astfel de blocuri
- La final, se face flatten pentru a transmite valorile vizuale către un MLP pentru a lua decizii

Batch Normalization

- Pe măsură ce modelul își ajustează greutatea, distribuția valorilor de ieșire ale unui strat (adică valorile trimise ca intrare pentru următorul strat) se schimbă
- La începutul antrenării, un strat poate produce valori cu o anumită medie și dispersie
- După câteva epoci de antrenare, ajustarea greutăților poate face ca aceste valori să devină mult mai mari sau mai mici
- Această variație face ca următoarele straturi ale rețelei să fie nevoite să se „re-adapteze” constant la noile distribuții ale datelor, încetinind procesul de învățare
- Batch Normalization ajută ca straturile unei rețele să învețe să dea ca ieșire date de o distribuție consistentă, practic să nu difere distribuțiile între epoci așa mult

Batch Normalization

- Batch Normalization ajută ca straturile unei rețele să învețe să dea ca ieșire date de o distribuție consistentă, practic să nu difere distribuțiile între epoci așa mult

Original Kernel (Values)

12.48	8.83	7.65
11.21	4.94	17.33
7.28	7.0	9.93

Batch Normalized Kernel

1.48	0.16	-0.26
1.02	-1.24	3.22
-0.4	-0.5	0.56

Batch Normalization

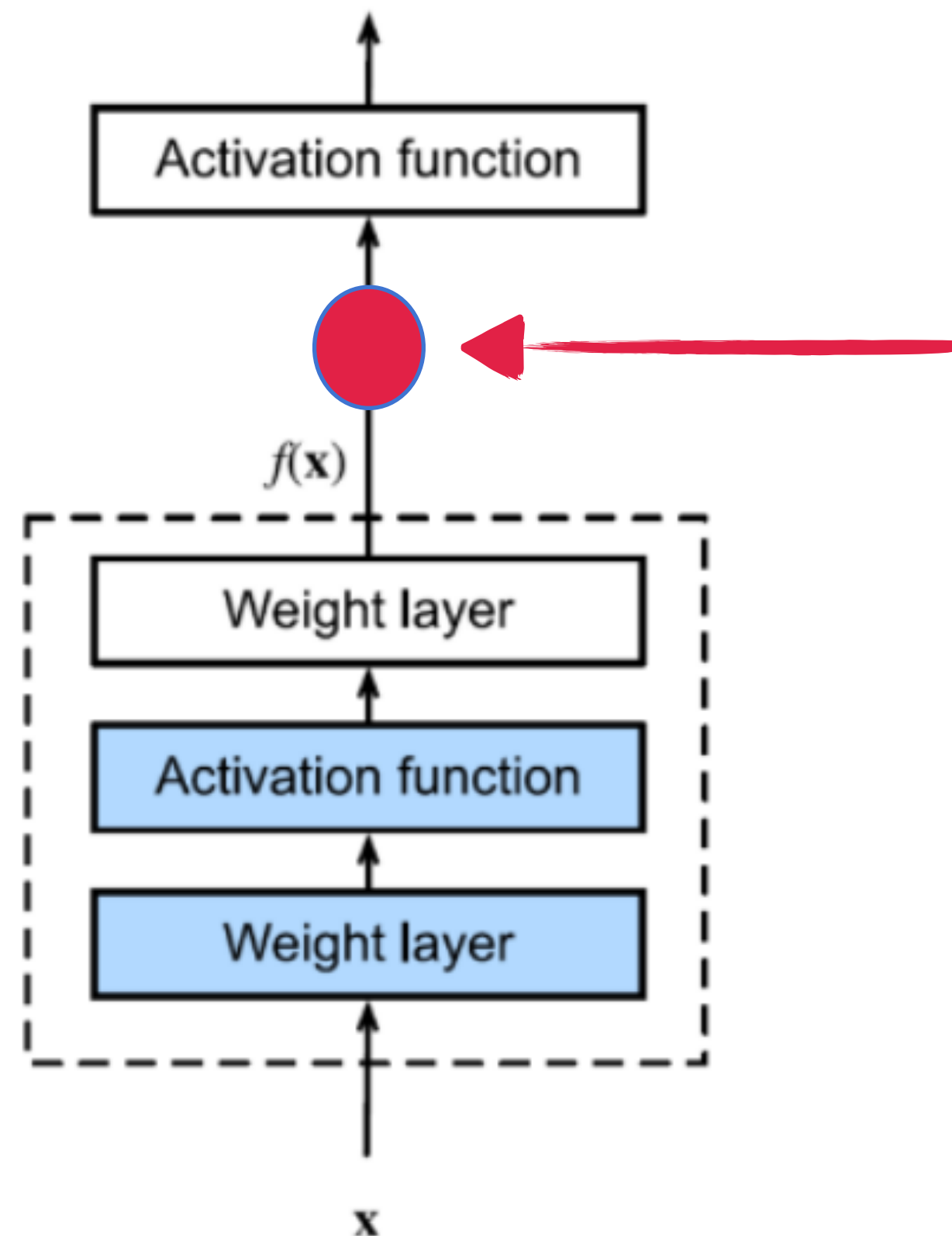
- Batch Normalization ajută ca straturile unei rețele să învețe să dea ca ieșire date de o distribuție consistentă, practic să nu difere distribuțiile între epoci așa mult

```
net = nn.Sequential(  
    nn.Conv2d(1, 6, kernel_size=5), nn.BatchNorm2d(6), nn.Sigmoid(),  
    nn.AvgPool2d(kernel_size=2, stride=2),  
    nn.Conv2d(6, 16, kernel_size=5), nn.BatchNorm2d(16), nn.Sigmoid(),  
    nn.AvgPool2d(kernel_size=2, stride=2), nn.Flatten(),  
    nn.Linear(256, 120), nn.BatchNorm1d(120), nn.Sigmoid(),  
    nn.Linear(120, 84), nn.BatchNorm1d(84), nn.Sigmoid(),  
    nn.Linear(84, 10))
```

- `nn.BatchNorm2d(n)`, unde `n` este numărul de canale de ieșire din stratul convoluțional anterior, este folosit după un strat convoluțional
- `nn.BatchNorm1d(m)`, unde `m` este numărul de neuroni de ieșire din stratul liniar anterior, este folosit după un strat liniar

Efectul straturilor asupra învățării

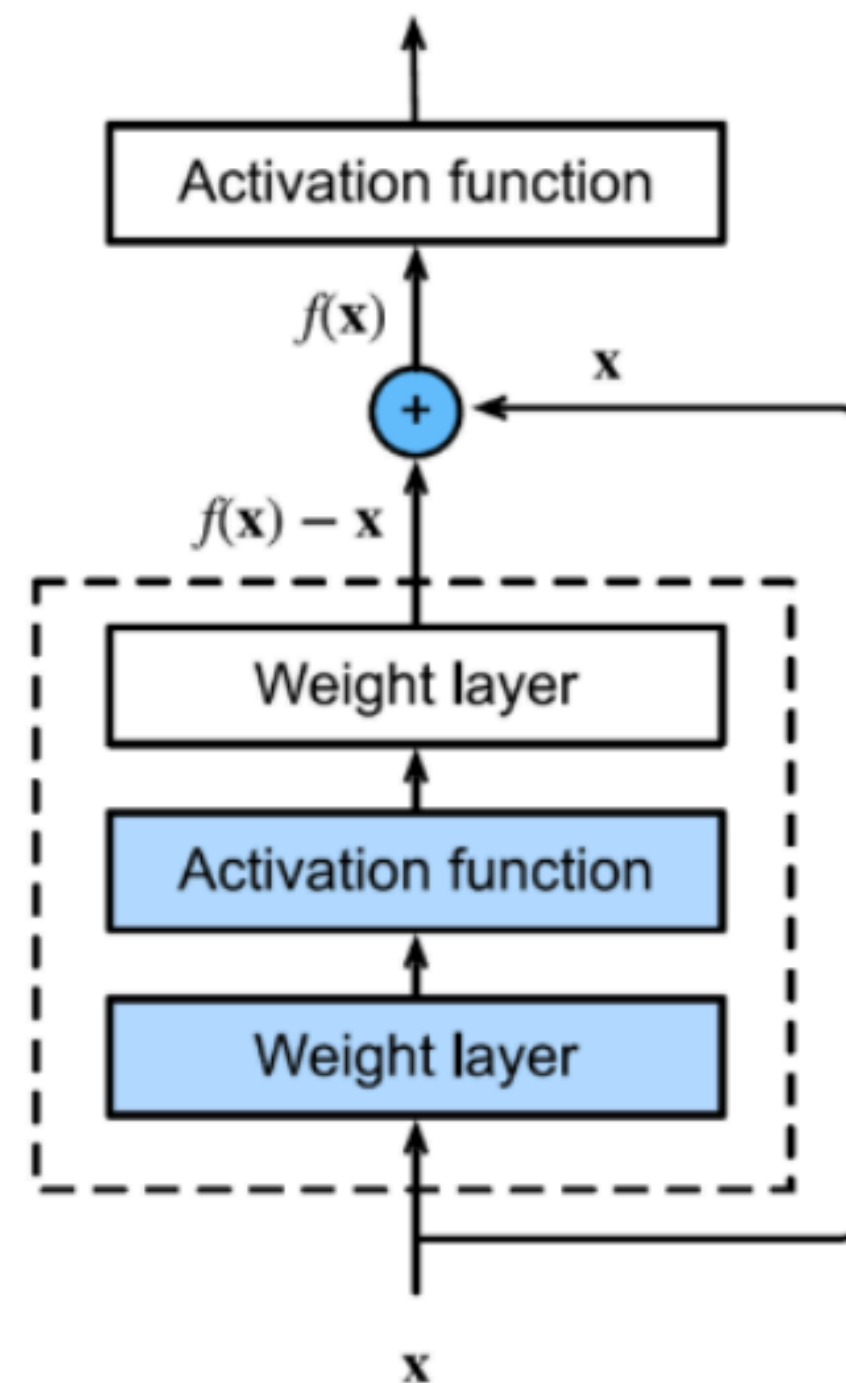
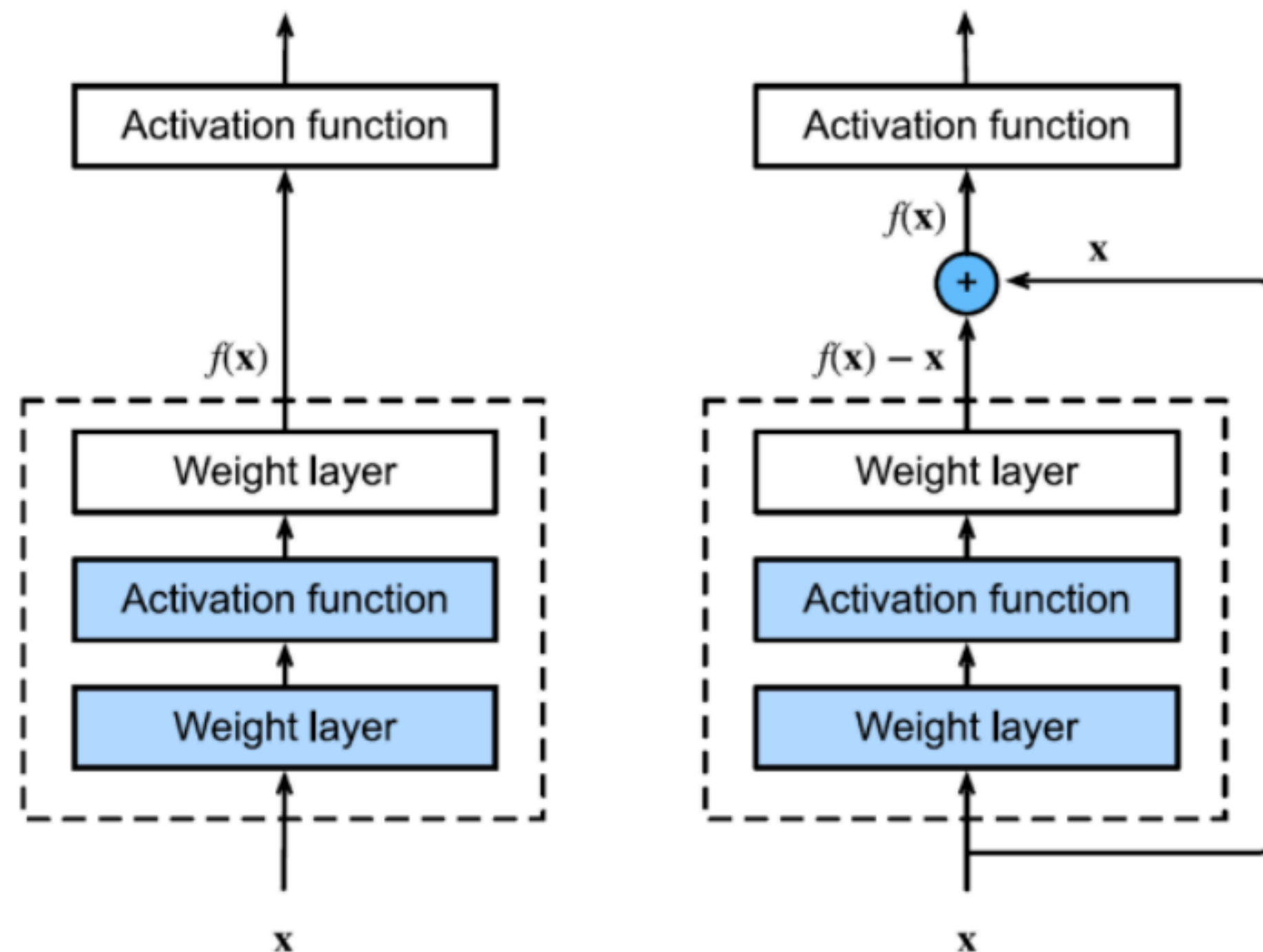
- Cu cât avem mai multe straturi, cu atât se poate ca modelul să învețe mai greu
- Asta pentru că există riscul să se uite informații din straturile anterioare



- În acest punct, se poate să se fi uitat detalii importante din X
- Și oricum, acest strat se poate sau nu să fie util, și atunci doar adaugă complexitate în plus

Conexiunile reziduale

- Conexiunile reziduale rezolvă aceste probleme, oferind modelului opțiunea de a „sări peste” un strat dacă este necesar



- Dacă stratul nu poate învăța nimic nou, conexiunea reziduală îi permite să lase ieșirea „aproape neschimbată”
- Dacă stratul poate învăța ceva util, adaugă această informație la ce știe deja despre X

Conexiunile reziduale (Arhitectura ResNet)

```
class Residual(nn.Module):
    """The Residual block of ResNet."""
    def __init__(self, input_channels, num_channels,
                 use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.Conv2d(input_channels, num_channels,
                                kernel_size=3, padding=1, stride=strides)
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.conv2 = nn.Conv2d(num_channels, num_channels,
                                kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(num_channels)
        if use_1x1conv:
            self.conv3 = nn.Conv2d(input_channels, num_channels,
                                    kernel_size=1, stride=strides)
        else:
            self.conv3 = None

    def forward(self, X):
        Y = nn.ReLU()(self.bn1(self.conv1(X)))
        Y = self.bn2(self.conv2(Y))
        if self.conv3:
            X = self.conv3(X)
        Y += X
        return nn.ReLU()(Y)
```

Aici se adaugă X la ce a fost deja procesat, deci e o conexiune reziduală

That's it for today!

That's it for today!

Laboratory 9 – Exercises

1. Define an input tensor of shape $(1,1,7,7)$ with random values. The two-dimensional convolutional layer should have as arguments: $c_in = 1$, $c_out = 1$, $padding = (2,1)$ and $stride = (3,3)$. Set the $kernel_size$ argument such that the output shape of the convolutional operation to be $(1,1,3,2)$.
2. Classify the SVHN dataset (32×32 images, 10 classes, 73257 training images and 26032 testing images) using ResNet architecture by applying only two modules of residual blocks. Train the model for 10 epochs with $lr = 0.9$ and $batch_size = 256$. Plot the training and validation accuracy.
3. Classify the SVHN dataset (32×32 images, 10 classes, 73257 training images and 26032 testing images) using the following convolutional neural network architecture. Divide the training dataset as follows: 30000 validation images and 43257 training images. Train your model for 5 epochs using a learning rate of 0.05 and a batch size of 256. Define a class for implementing the Convolutional Block. Conv2d, 32, 3×3 means 32 output channels and 3×3 kernel size. Use padding, if necessary.

